

University of Wollongong
School of Computing and Information Technology
CSCI318 Software Engineering Practices & Principles
Spring 2026

Group Project (40 marks)

- You need to form a group of **3 people** and register your group using a form on the Moodle site of the subject.
- Structure your group, allocate roles and responsibilities to your members and organize **weekly meetings**. Each group must have **at least one meeting** per week.
- Make sure your group **make progress on the project every week**

SUBMISSION

1. Project Plan (**2 marks**): **Week 4 Lab**.
 - a. One member from each group presents the progress of their group in **5 minutes**
 - b. The progress report presentation should contain:
 - i. Introduce their group members and roles
 - ii. Preliminary findings of all the tasks such as details about the projects, software applications and testing tools your group selects to study.
 - iii. Work plans in the form of a Work Breakdown Structure (e.g. tasks, sub-tasks, who perform what, outcomes, etc.)
 - iv. Meeting plans, e.g. how many meetings per week, date/time and location of meetings.
2. Progress Report (**3 marks**): submit to Moodle in **Week 8 (submission deadline will be set on Moodle)**
 - a. A short video (10 minutes maximum) presents the progress of your group
 - b. The progress report presentation (using slides) should contain:
 - i. Reporting work completed and work in progress.
 - ii. Presenting plans for the remaining of the project.
 - iii. Member contribution ratings so far.
3. Weekly meeting minutes (**3 marks**): submitted to Moodle **every week** from **Week 3 – Week 11 inclusively**) should cover at least the following:
 - Genuine group meeting records: agendas and meeting minutes which include at

least the following: meeting date, attendance, progress reports, discussion summaries, and action plans/items. In particular, *report what has been done in the previous week, and what has been planned in the next week.*

4. Final deliverables (**27 marks**): submitted to Moodle by **COB Monday Week 12 (tentative)**

The project report (PDF format), a zip file of a software repository including the source code, specification files, README file and other artefacts (if any) must be submitted to the Moodle site of the subject by one member of your group.

5. Final presentation (**5 marks**): delivered in **Week 12 Lab or Lecture**

The presentation guidelines (including a presentation schedule) will be provided in a separate document before the presentation week.

IMPORTANT NOTES:

- **Generative AI Use**

- Students can use **Generative AI tools and agents to assist with software development activities**, including requirements engineering, domain modelling, architecture design, implementation, testing, and software documentation within the project repository.
- **The project report and presentation must be authored by the student team and must not be generated by Generative AI.** The report should represent the team's own analysis, design decisions, justifications, reflections, and lessons learned. The project report may **reference or reproduce artefacts from the submitted software repository**, including specifications, prompts, architecture documents, and source code **that were generated or assisted by AI tools or agents**, provided their origin is clearly identified where appropriate.

- **Member contribution for the whole project (with each member's signature)**

- On the cover page of your group's report, you need to provide rating for the contribution of each team member and a **detailed explanation** of what the team member did for the project to justify the rating.
- Everyone in the team should write a statement ***"I agree with my group member contribution report and ratings"*** and **insert their signature next to the statement.**
- The individual contribution of each team member is assessed by all the other members.
- The rating scale can be a **specific percentage number** (e.g. 40%, 60%, 80%, etc.).
- Alternatively, it can be rated into one of the three scales: "contributed", "very little", and "almost no contribution". For a team member who has "contributed", he/she will receive 100% of the group mark; for a team member who contributed "very little", he/she will receive 50% of the team mark; for students who made "almost no contribution", he/she will receive 0 marks for the entire group project. Your tutor/lecturer may make adjustment to this marking criterion based on practical situations.

PROJECT SPECIFICATION

Project Goal

The objective of this project is to design and implement an **AI-powered cloud-native application** within a realistic business domain using **AI-assisted software engineering practices**. The project should demonstrate the practical application of software engineering principles and patterns discussed in the CSCI318 lectures, including Domain-Driven Design (DDD), microservices architecture, event-driven architecture, stream processing, and agentic AI patterns.

Business Domain

Your project team will determine the specific business domain (e.g., travel booking, event registration, online retailing, healthcare, logistics). When selecting your domain, consider its suitability for demonstrating the required architectural patterns and ensuring a manageable scope for the project timeline. The application must include at least four distinct bounded contexts, each implemented as a separate microservice. These microservices should communicate via REST requests and/or asynchronous messaging.

Key Architectural Requirements

- **Microservices:** At least four (4) bounded contexts, each implemented as a microservice.
- **Event-Driven Architecture:** Adopts an event-driven architecture for inter-service communication.
- **Stream Processing:** Supports real-time data queries using stream processing techniques.
- **Agentic AI:** Integrates LLM-based agentic components for specific use cases.

Implementation Framework

- This prototype is to be implemented using the Spring Boot framework with the following mandatory dependencies: Spring Web, Spring Data JPA, H2 (for local development/testing; teams may propose alternatives if justified), Spring Cloud Stream and LangChain4j
- Java Version: **JDK 21**
- Messaging Platform: Apache Kafka will be used as the messaging platform for event-driven communication.
- LLM Model Selection: The selection of a specific LLM model is at the discretion of the project team. Consider options based on accessibility and feasibility.
- Additional Dependencies: Additional dependencies beyond those listed above may be used, but explicit permission must be sought from the subject coordinator. This ensures alignment with learning objectives and helps manage project complexity.

Detailed Requirements and Project Report Structure

The final project deliverable includes a professional report and a software repository.

Project Report Structure:

Cover page

- The application name (determined by your project group).
- The project group's ID and name (if any), and all group members.
- Accountability information (member contribution). See above.

Abstract

- This is an executive summary of the application, in one or two paragraphs.
- A link to the GitHub repository of this project is included.

Table of Contents

Section 1. Introduction

- Provides introductory information and a concise overview of the application.
- Address the business case for this application: What problem does it solve? What are its primary objectives (e.g., business needs, stakeholders' expectations)?
- Describe the system's core capabilities.
- Explain how these system capabilities meet the stated objectives, specifically highlighting how AI empowers the business and adds value.

Section 2. Analysis, Requirements and Domain Modeling

1. Functional Requirements.

- Identify and clearly state **at least 12 functional requirements**.
- For each requirement, explain its relevance to the objectives (e.g., business need, stakeholders' expectations) stated in Section 1.
- Specify these requirements using use cases and illustrate them with UML activity diagrams.

2. Real-time Analysis Requirements

- **At least two (2)** of the identified user stories must involve real-time analysis based on streaming data processing.

3. Agentic AI Requirements

- **At least two (2)** of the identified user stories must involve the use of an agentic AI component.

4. Domain Modeling.

- Identify the domain classes for your chosen business domain, and structure those domain classes into sub-domains.
- Develop a comprehensive UML domain model diagram.
- Justify how these domain classes and sub-domains accurately reflect the problem domain (i.e., business domain) and align with the concept of bounded contexts, based on your reasonable assumptions.

Section 3. Design, Architecture and Implementation

This section should provide a detailed explanation of your application's design and implementation.

1. Microservice Architecture:

- Present an architectural diagram for your microservice application.
- This diagram should clearly depict all services, their inter-service communication mechanisms (e.g., via REST requests, asynchronous messaging), and data flows. The diagram must be indicative and facilitate a clear understanding of your system's overall structure.

2. Layered Architecture:

- Illustrate the layered architecture adopted for each microservice. Each service is expected to include a presentation layer, a service layer, a domain layer, and an infrastructure layer.
- Explain the purpose and responsibilities of each layer.
- Demonstrate the alignment of these architectural layers with the namespace (i.e., package) structure in your software project's codebase of each service.

3. Domain-Driven Design Patterns:

- Discuss the design patterns applied to your domain classes, specifically focusing on entities, value objects, aggregates, domain events, and domain services.
- Utilise the Domain-Driven Design (DDD) principles from the CSCI318 lectures to explain why a particular domain class belongs to a specific pattern.
- Include relevant sample code snippets to support and clarify your explanations.

4. Data Persistence in Microservices:

- Illustrate your database design.
- Clearly define the data ownership for each microservice (e.g., using a table to illustrate which microservice owns which data entities).

5. Event-Driven Architecture and Stream Processing:

- Explain the design and implementation of your event-driven architecture.
- Detail what stream processing queries are considered (e.g., using SQL-like language) and how they are implemented to achieve real-time analysis.
- Present sample code to illustrate your event publishing/consuming and stream processing logic.

6. Agentic AI Component:

- Provide a detailed description of the agentic design patterns used.
- Explain how the LLM models are integrated into your application and how their APIs are utilised.
- Present sample code to demonstrate the implementation of your agentic AI components.

7. Other Important Aspects:

- Include any other significant aspects of your design, architecture, or implementation that contribute to the project's success or demonstrate advanced understanding.

Section 4. Instruction and User Story Examples

1. Configuration and Running Instructions:

- Provide clear, step-by-step instructions on how to configure and run the entire software project, including Spring Boot applications, Apache Kafka, and any LLM API configurations.

2. User Story Input and Output:

- For each implemented user story, present at least one sample input and its corresponding output.
- As a full-fledged frontend is not a required component of this group project, external users will primarily communicate with the microservices by calling their REST API endpoints.
- Consider implementing a lightweight client (e.g., a simple command-line script or a Postman collection) to automate some REST requests or pre-populate the database, especially for demonstrating stream processing scenarios.

Section 5. AI-Driven Development Process

1. Human-AI Collaborative Development

- Describe **how team members interacted with AI tools and agents** throughout the software development lifecycle, including requirements engineering, domain modelling, architecture design, implementation, testing, and documentation.
- Provide **clear references to related artefacts in the software project repository** (e.g., specifications, prompts, design documents, source code, tests, agent workflows) to support and demonstrate the collaborative development process.
- Include **representative examples** where AI-generated proposals were accepted, rejected, or modified by the team, and explain the **rationale** behind those decisions.

2. Lessons Learned

- Reflect on the strengths and limitations of AI-assisted software engineering.
- Discuss the impact of AI on productivity, software quality, architectural decision-making, and team collaboration.
- Use concrete examples in the context of this project to support your statements.

References

- A correct reference format (e.g., in the Harvard style) must be used.

README.md File:

A README.md file must be included in the root directory of your software project. This file should

contain:

- Configuration and running instructions (as detailed in Section 4.1).
- Sample inputs to demonstrate the use cases (outputs are not necessary in the README)

Important Note: Keep the GitHub repository private until submitting the project deliverables. Don't share the repository or code with other groups.

-----END OF SPECIFICATION-----